NetStar Ticketsystem – Teil 2: Bonusaufgaben (Aufgabenblatt)

Oct 12, 2026 · @Christopher Partridge

Worum es geht

Teil 2 ist für alle, die Teil 1 fertig haben: A1 bis B7 laufen. Ihr dokumentiert, erweitert und testet euer Ticketsystem. Dafür sucht ihr euch Aufgaben aus drei Bereichen aus. Alles baut auf Collections, Lambdas und Streams auf.

Kopiert vorher euren Projektordner als Sicherung.

Was neu für euch ist, dürft ihr nachschlagen, im Internet suchen oder eine KI fragen. Lasst euch Dinge und Fehlermeldungen erklären, aber den Code schreibt ihr selbst. Wichtig ist, dass ihr jede Zeile versteht, die ihr schreibt.

Nr.	Aufgabe	Bereich	Richtwert
~~D1~~	~~Javadoc schreiben und erzeugen~~	~~Dokumentation~~	~~40 min~~
D2	Klassendiagramm zeichnen	Dokumentation	40 min
D3	Testprotokoll führen	Dokumentation	30 min
D4	README mit Begründung der Collections	Dokumentation	30 min
E1	Filter kombinieren	Erweiterung	30 min
E2	Sortier-Menü	Erweiterung	30 min
E3	Statistik	Erweiterung	30 min
E4	Bearbeiter-Ranking	Erweiterung	30 min
T1	Selbsttest-Klasse	Testen	40 min
T2	Fehlersuche	Testen	40 min

Wählt mindestens eine Aufgabe aus jedem Bereich, die Reihenfolge ist frei. Wenn ihr nicht wisst, womit ihr anfangen sollt: D1, E1, T1, T2. Die Zusatzaufgaben Z1 bis Z4 aus Teil 1 zählen ebenfalls als Erweiterungen. Es muss nicht alles geschafft werden.

Bereich 1: Dokumentation

Guter Code ist dokumentierter Code. Legt alle Ergebnisse dieses Bereichs in einem Unterordner `doku` in eurem Projekt ab.

Nr.	Das tut ihr	Das entsteht
D1	Schreibt Javadoc-Kommentare für die Klassen `TicketSystem` und `Auswertung` und für alle `public`-Methoden. Jede Methode bekommt einen kurzen Satz, ein `@param` pro Parameter und ein `@return`, wenn sie etwas zurückgibt. Erzeugt daraus die HTML-Dokumentation und öffnet `doku/index.html` im Browser.	Ordner `doku` mit der erzeugten HTML-Dokumentation, die Javadoc-Kommentare im Quelltext
D2	Zeichnet ein UML-Klassendiagramm von `Ticket`, `TicketSystem`, `Auswertung` und `Main`. Eingetragen sind alle Attribute mit Typ (bei den Collections mit dem Typ der Elemente), die `public`-Methoden mit Sichtbarkeit und die Beziehungen mit Multiplizitäten. Das Werkzeug ist frei: draw.io, PlantUML oder Papier mit Foto.	`doku/klassendiagramm.png` (oder `.pdf`, `.jpg`)

D3	Führt ein Testprotokoll mit mindestens 8 Testfällen. Spalten: Nr., Menüpunkt oder Methode, Eingabe, erwartetes Ergebnis, tatsächliches Ergebnis, bestanden (ja/nein). Mindestens 2 Testfälle sind Fehlerfälle, zum Beispiel leere Warteschlange oder Text statt Zahl bei der Priorität. Das tatsächliche Ergebnis tragt ihr nach dem echten Ausführen ein, nicht aus dem Kopf.	`doku/testprotokoll.md` (oder Word, PDF)
D4	Schreibt eine `README.md`: Was macht das Programm? Wie kompiliert und startet man es? Was tut jeder Menüpunkt? Dazu eine Tabelle "Welche Collection wofür und warum" (Collection, Aufgabe im Programm, warum diese und keine andere) und ein Beispielablauf mit eingefügter Konsolenausgabe.	`README.md` im Projektordner

So sieht ein Javadoc-Kommentar aus (ohne Umlaute, denn es bleibt Quelltext):

```
/**
 * Legt ein neues Ticket mit der naechsten freien Id an.
 *
 * @param titel      Titel des Tickets
 * @param prioritaet 1 (kritisch) bis 4 (niedrig)
 * @return das neu angelegte Ticket
 */
public Ticket anlegen(String titel, String kunde, String bearbeiter, int prioritaet) {
```

Tipps:

- Erzeugen könnt ihr die Dokumentation im Projektordner mit `javadoc -d doku -encoding UTF-8 TicketSystem.java Auswertung.java`. Schaut euch die Warnungen an: Jede Warnung heißt, dass ein Kommentar fehlt oder unvollständig ist. Auch eine Klasse ganz ohne eigenen Konstruktor löst eine Warnung aus (Standardkonstruktor ohne Kommentar): Schreibt dann einen eigenen, leeren Konstruktor mit Kommentar.
- Typen mit spitzen Klammern schreibt ihr in Javadoc als `{@code List<Ticket>}`, sonst gibt es Fehler.
- Bei D2 hilft die Frage: Wer kennt wen? `TicketSystem` und `Auswertung` benutzen `Ticket`. `Main` benutzt `TicketSystem` und `Auswertung`.

Bereich 2: Erweiterungen

Ihr ergänzt `Auswertung` und `Main` um drei neue Menüpunkte: 8 (Filter), 9 (Sortieren) und 10 (Statistik). Die Prüfwerte gelten direkt nach dem Programmstart mit den unveränderten Beispieldaten.

Nr.	Das tut ihr	So prüft ihr es
E1	Filter kombinieren (Menüpunkt 8). Schreibt in `Auswertung` zwei Methoden: `static Predicate<Ticket> vonBearbeiter(String name)` gibt ein Lambda zurück, und `List<Ticket> filtern(Predicate<Ticket> filter)` liefert alle Tickets, die der Filter durchlässt. Kombiniert eure vorhandenen `istOffen` und `istKritisch` mit `and`, `or` und `negate`, ohne neue Schleifen: (a) offen UND kritisch, (b) offen UND NICHT kritisch, (c) kritisch ODER von Anna Berger. Menüpunkt 8 gibt (a) bis (c) aus und fragt danach nach einem Bearbeiter. Ausgegeben werden dessen offene Tickets.	(a) #2, #5. (b) #1, #3, #7, #9. (c) #1, #2, #3, #5, #7, #10. Tom Weber: #2. Unbekannter Name: 0 Tickets und kein Absturz

E2	Sortier-Menü (Menüpunkt 9). Legt in `Auswertung` eine `Map<String, Comparator<Ticket>>` an mit den Schlüsseln `id`, `prioritaet`, `titel` und `kunde`. Bei Gleichstand entscheidet immer die Id (`thenComparing`). Schreibt `List<Ticket> sortiert(String kriterium, boolean absteigend)`. Absteigend heißt: die komplette Reihenfolge mit `reversed()` umkehren. Menüpunkt 9 fragt nach dem Kriterium und nach "absteigend (j/n)" und gibt die sortierte Liste aus. Ein unbekanntes Kriterium gibt eine Meldung aus.	`titel`: #5 steht oben, #6 unten. `prioritaet` absteigend: #8, #3, #6, #4, #9, #7, #1, #10, #5, #2. `farbe`: Meldung statt Absturz
E3	Statistik (Menüpunkt 10, erster Teil). Schreibt in `Auswertung`: (a) eine `Map<Boolean, List<Ticket>>` mit `Collectors.partitioningBy(istOffen)`, (b) eine Methode, die die Titel aller Tickets mit einem bestimmten Status mit `Collectors.joining(", ")` zu einem Text verbindet, (c) die Prioritätsstatistik mit `mapToInt` und `summaryStatistics()`. Menüpunkt 10 gibt alle drei aus.	(a) offen 6, nicht offen 4. (b) für `IN_BEARBEITUNG`: WLAN im Besprechungsraum langsam, Server-Festplatte fast voll. (c) min 1, max 4, Durchschnitt 2,3, 10 Tickets (je nach Systemsprache mit Komma oder Punkt)
E4	Bearbeiter-Ranking (Menüpunkt 10, zweiter Teil). Welcher Bearbeiter hat die meisten offenen Tickets? Filtert die offenen Tickets, zählt sie pro Bearbeiter mit `groupingBy` und `counting` und sucht in `entrySet().stream()` den Eintrag mit dem größten Wert (`max` mit `Map.Entry.comparingByValue()`). Das Ergebnis ist ein `Optional`. Gibt es keine offenen Tickets, gebt ihr "Keine offenen Tickets." aus.	Anna Berger (3). Erledigt ihr mit Menüpunkt 3 sechsmal hintereinander alle offenen Tickets, kommt "Keine offenen Tickets.". Bei Gleichstand ist jeder der Gleichauf-Bearbeiter richtig

Tipps:

- Lambdas schreibt ihr aus, keine Methodenreferenzen mit `::`. `Map.Entry.comparingByValue()` ist ein normaler Methodenaufruf und erlaubt.
- Gebt jedes Zwischenergebnis mit `System.out.println` aus, bevor ihr die nächste Stream-Operation anhängt.
- Beim ersten `Comparator.comparing` braucht das Lambda eine Typangabe: `(Ticket ticket) -> ...`.
- Verlangt ein Menüpunkt eine Eingabe, die nichts findet (unbekannter Name, unbekanntes Kriterium), muss das Programm weiterlaufen.

Bereich 3: Testen und Fehlersuche

Nr.	Das tut ihr	Das entsteht, so prüft ihr es
T1	Schreibt eine eigene Klasse `Selbsttest` mit `main`-Methode, die euer Programm automatisch prüft, ohne Testframework. Eine Hilfsmethode `pruefe(String name, Object erwartet, Object tatsaechlich)` gibt `OK` oder `FEHLER` samt beiden Werten aus. Am Ende steht "x von y Tests bestanden". Schreibt mindestens 8 Tests mit den Prüfwerten aus Teil 1, davon mindestens 2 für `TicketSystem` (Reihenfolge der Queue, Rückgängig). Für jeden Test am `TicketSystem` braucht ihr ein frisches System. Baut danach absichtlich einen falschen Erwartungswert ein und schaut, ob er als `FEHLER` erscheint. Dann nehmt ihn wieder heraus.	`Selbsttest.java`. `java Selbsttest` gibt eine Zeile pro Test aus, alle sind `OK`, am Ende steht z. B. "15 von 15 Tests bestanden". Der absichtlich falsche Test wurde als `FEHLER` gemeldet
T2	Ein Kollege hat die Datei `Fehlersuche.java` geschrieben. Legt sie neben `Ticket.java` und `Beispieldaten.java` in euer Projekt. Sie kompiliert, aber jede der 6 Methoden hat einen Fehler. (1) Startet `java Fehlersuche` und notiert für jede Methode das Symptom. (2) Findet die Ursache und erklärt sie in einem Satz. (3) Korrigiert die Methode, was sie tun soll steht im Kommentar darüber. (4) Tragt alles in eine Tabelle ein: Methode, Symptom, Ursache, Korrektur.	Korrigierte `Fehlersuche.java` und `doku/fehlersuche.md`. Nach der Korrektur: Methode 1 gibt 6 Titel aus, Methode 2 "Anzahl offen: 6" und 6 Titel, Methode 3 "8" Tickets, Methode 4 die Ids [2, 5, 10], Methode 5 den Wert 2.3, Methode 6 "VPN-Verbindung bricht ab" und "Keine Tickets"

Tipps zu T1:

- `anzahlOffen()` liefert ein `long`. `Objects.equals(6, 6L)` ist aber `false`, weil `Integer` und `Long` verschiedene Typen sind.
- Zahlen mit Nachkommastellen vergleicht man nie mit `==`, sondern mit einer kleinen Toleranz, zum Beispiel `Math.abs(erwartet - tatsaechlich) < 0.0001`.
- Zwei Versionen der Methode (`pruefe(String, double, double)` und `pruefe(String, Object, Object)`) lösen beide Probleme.

Tipps zu T2:

- Lest bei einer Fehlermeldung zuerst die erste Zeile: Sie nennt den Fehlertyp. Dann sucht die Stelle im Code, an der er auftritt.
- Die Fehler sind von unterschiedlicher Art: Bei manchen kommt eine Fehlermeldung, bei manchen ein falsches Ergebnis und bei einem passiert gar nichts. Schaut euch auch an, was Teil 1 über Streams gesagt hat.

Regeln und Abschluss

- Recherche und KI sind erlaubt: Dokumentation nachlesen, Begriffe und Fehlermeldungen erklären lassen, Ideen besprechen. Den Code und die Texte schreibt ihr selbst und könnt sie erklären.
- Lambdas schreibt ihr aus, keine Methodenreferenzen mit `::`.
- Im Quelltext (auch in Javadoc-Kommentaren) keine Umlaute und kein ß (ae, oe, ue, ss), sonst zeigt die Windows-Konsole Zeichensalat. In `README.md` und den Protokollen sind Umlaute erlaubt.
- `Ticket.java` und `Beispieldaten.java` bleiben unverändert. Arbeitet in der Kopie eures Projekts.
- Prüft eure Ergebnisse mit den Prüfwerten in den Tabellen. Ein Programm, das nicht kompiliert, ist nichts wert: kompiliert nach jeder Teilaufgabe.
- Es muss nichts abgegeben werden, es ist eine Übung. Am Ende schauen wir uns den entstandenen Code und die Ergebnisse gemeinsam an. Wer möchte, schickt der Lehrkraft sein Ergebnis.

© Christopher Partridge